

DMND



SMART CONTRACTS AUDIT

DrCrypto.ca - January 2026
Revision (v2): February 2026

Disclaimer

This report is intended solely for informational purposes and reflects the analysis and opinions of the author based on the data provided. It should not be construed as legal, financial, or professional advice. The information contained herein is not a substitute for the advice of qualified legal, financial, or other professional advisors. The author does not assume any liability or responsibility for any actions taken based on the information provided in this report. Readers are encouraged to seek the appropriate professional counsel before making any decisions related to cryptocurrency investments or any related matters.

DMND

Smart Contracts Audit

Preface

This audit is of the DMND₁, DMND₅, and DMND₁₀ tokens & vault contracts that were deployed to the Ethereum mainnet on January 31, 2026. This was manually audited as well as reviewed with other tools.

DMND1 VAULT: 0xCc30bCfa5AED71d23d2ad7A7E51c2052dfa4Ec77

DMND1 TOKEN:

0xb9E7620299363894A864bec115FbE1a2e513c74C

DMND5 VAULT:

0xA2d91bA2DE7da4919d3840Be180406d420CA364f

DMND5 TOKEN:

0x99742587462a8a972103FebaB04B45DdF2c6fD2a

DMND 10 VAULT:

0x2e4C49455c203a8CE00123Aa85429336f23481e9

DMND10 TOKEN:

0x117Aa6BB9531a5edC9B82270970E91458C816E73

REVISION (v2):

Deployment of **ConvertFeesGuard** on Ethereum mainnet at `0x74e8b3d931ce4a1f46518510ae4d381ac535276b` (deployed Feb 19, 2026)

This revision updates the holistic audit to incorporate the newly deployed guard contract that enforces the “minOut” post-swap invariant **without requiring redeployment** of the DMND1 Token/Vault.

Executive Summary

System components (current mainnet architecture):

1. **DMND1Token** — fixed-supply ERC-20, sell-side fee accrual, public `convertFeesToUSDC(minOut, deadline)` swap-to-vault.
2. **DMND1Vault** — USDC reserve, proportional redemption with burn.
3. **ConvertFeesGuard** — permissionless helper that wraps fee conversion and enforces **vault USDC delta $\geq$ minOut**.

Overall posture (v2):

The system remains minimal, ownerless after one-time setup, and economically coherent. The new **ConvertFeesGuard** improves robustness of fee conversion by verifying **what the vault actually received**, providing an additional safety property that previously existed only as “polish.”

Key v2 improvement:

-  **Stronger minOut enforcement for conversions:**
`ConvertFeesGuard.guardedConvert()` checks vault USDC balance before/after and reverts if `delta < minOut`.

Residual notes:

- No new privileged authority is introduced.

System Architecture and Flow (with Guard)

Fee accrual (Token)

- Fees accrue as DMND in DMND1Token (on sells into the pair).

Fee conversion (two ways)

A) Direct conversion (existing path):

- Caller invokes `DMND1Token.convertFeesToUSDC(minOut, deadline)`.
- Router executes swap and sends USDC to vault.

B) Guarded conversion (recommended ops path):

- Caller invokes `ConvertFeesGuard.guardedConvert(token, minOut, deadline)`.
- Guard reads `USDC.balanceOf(vault)` before.
- Calls `token.convertFeesToUSDC(minOut, deadline)`.
- Reads `USDC.balanceOf(vault)` after.
- Enforces: `after - before >= minOut` or reverts entire tx.

Redemption (Vault)

- Holder calls `DMND1Vault.redeem(amountDMND, minUSDCOut)`.
- Vault computes payout from current USDC balance and total supply, burns DMND, transfers USDC.

Security Properties Added

✓ P1 — “Actual vault receipt” invariant

Even if a router/token path behaves unexpectedly, the guard validates **real USDC received by the vault**, not just router-side expectations.

✓ P2 — Atomicity

If the guard’s final `require(actual >= minOut)` fails, the transaction reverts, rolling back:

- the swap,
- token state changes,
- and any intermediate transfers.

✓ P3 — No privileged control

Guard has no owner, no admin, no token custody, no approvals—purely a wrapper and a check.

Inter-Contract Interaction Review (v2)

ConvertFeesGuard ↔ DMND1Token

- Guard calls `token.vault()` and `token.usdc()` to resolve addresses.
- Guard then calls `token.convertFeesToUSDC(minOut, deadline)`.

Reentrancy:

- Guard itself has no state mutation that needs a reentrancy guard.
- Token conversion is already guarded by `nonReentrant` and a swap sentinel in the token code you shared previously.
- Even in worst-case, the guard performs only `balanceOf` reads before/after.

ConvertFeesGuard ↔ Vault

- Guard never calls the vault contract directly.
- It only observes `USDC.balanceOf(vault)`.
- Vault's redemption system remains unaffected except that it benefits from more reliable conversion outcomes.

Final Summary (v2)

- The DMND1 Token + Vault system remains secure and minimal as reviewed.
- The newly deployed **ConvertFeesGuard** at `0x74e8b3d931ce4a1f46518510ae4d381ac535276b` is well-designed for its purpose and meaningfully improves safety by enforcing that **the vault actually received at least minOut** for conversions.
- No new critical/medium security vulnerabilities were introduced by this contract.
- The only meaningful caveat is that guard usage is **voluntary** unless you standardize it operationally; recommended practice is to route conversions through the guard.

DMND1 — Holistic Smart Contract Security Audit

Targets: `DMND1Vault` (USDC-backed redemption vault) and `DMND1Token` (fixed-supply ERC-20 with sell-side fee & fee conversion)

Language/Compiler: Solidity `^0.8.24` (native overflow checks)

Scope: Deployed code you pasted (no comparison to prior testnet builds)

Executive Summary

Overall posture:

The system is minimal, non-custodial, and **ownerless** after one-time wiring. The vault implements proportional redemptions with correct pre-burn accounting, while the token enforces a **sell-side fee** (on transfers to the AMM pair) and exposes a public function to **convert accrued DMND fees to USDC**, which are sent **directly** to the vault. Reentrancy surfaces are well controlled, and operational misconfiguration is mitigated with code-size checks and constructor guards.

Key outcomes (current code):

-  **Immutability & no privileged controls** after setup (token pair + vault linkage sealed).
-  **Sell-side fee** is correctly assessed on router-mediated sells (`to == pair` and sender not fee-exempt).
-  **Public fee conversion** routes `DMND` $\rightarrow$ (WETH?) $\rightarrow$ `USDC` $\rightarrow$ sent to the vault, with `minOut` and `deadline`.
-  **Proportional redemptions** use **pre-burn** totals; slippage guard protects users.
-  **Reentrancy**: token conversion and vault redemption are guarded; no external hooks are relied upon.

Open risks / recommendations (non-blocking):

-  **Defense-in-depth:** add `require(out >= minUSDCOut)` after swap (you already pass `minOut` to the router; this is a redundant on-chain proof using the vault's USDC delta).
-  **Custom errors** instead of string reverts (minor gas + clarity improvement).

Verdict:

From a security perspective, the contracts are sound and align with the stated design goals. Remaining suggestions are optional polish.

System Overview

Roles & trust boundaries

- **Users:** Hold DMND and can redeem for a proportional share of vault USDC at any time.
- **Anyone:** May call token's `convertFeesToUSDC(minOut, deadline)` to convert accrued DMND fees to USDC added to the vault.
- **Deploy-time actor:** May set the token's AMM pair once; may set the vault's token once. Both actions seal permanently.
- **Uniswap V2 router (trusted infra):** Used for swaps; the token assumes canonical behavior of `swapExactTokensForTokensSupportingFeeOnTransferTokens`.

Assets & flow

1. **DMND fees accrue** inside the token contract on sell transactions.
2. **Conversion** swaps accrued DMND to USDC using a configurable path (DMND→USDC or DMND→WETH→USDC) and sends USDC directly to the **vault**.
3. **Redemption:** A DMND holder burns tokens via the vault and receives `pro-rata(USDC)`.

Contract-by-Contract Review

DMND1Vault (USDC Reserve & Proportional Redemption)

Core storage & wiring

- USDC is **immutable** and validated in constructor for non-zero and `code.length > 0`.
- `token` is set **once** via `setToken`, restricted to an `_authorizedSetter` set at deployment; guarded by non-zero, `code.length > 0`, and a `try totalSupply()` interface probe; then sealed (`_authorizedSetter = 0` and `isTokenSet = true`).

Redemption logic

- `vaultBalance()` returns current USDC balance.
- `floorPerToken1e18() = vaultBalance * 1e18 / totalSupply` (view helper).
- `redeem(amountDMND, minUSDCOut)`:
 - Requires: token linked, `amount > 0`, `totalSupply > 0`, `vaultBalance > 0`.
 - Computes **pre-burn** payout: `usdcOut = amountDMND * vaultBalance / totalSupply`.
 - Slippage/dust guards: `usdcOut > 0` and `usdcOut >= minUSDCOut`.
 - Calls `token.burnFrom(msg.sender, amountDMND)` then `USDC.safeTransfer(msg.sender, usdcOut)`.
 - Emits `Redeemed(account, amountDMND, amountUSDC, newFloor1e18)`.

Security notes

- **Reentrancy:** `redeem` is `nonReentrant` and only calls your own token + USDC; both are non-callback flows (USDC canonical).
- **DoS:** No loops over user-controlled data; calculations are $O(1)$.
- **Privileged actions:** none after sealing `setToken`.

Assessment: ✓ Solid, non-custodial, and consistent with the proportional floor model.

DMND1Token (Fixed Supply, Sell-Side Fee, Fee Conversion)

Core storage & wiring

- Immutable infra: `vault`, `router`, `usdc`, `weth` (optional), `useWethRoute`, `feeBps` ($\leq 10\%$), `minConvertAmount`.
- **Exemptions:** `isFeeExempt[address(this)] = true`, `isFeeExempt[vault] = true`. **Router is not exempt**—critical for sell-fee correctness.
- **One-time pair wiring:** `setPair` requires caller is `_authorizedSetter`, enforces non-zero and `code.length > 0`, sets `uniswapPair`, marks `isPairSet = true`, then seals by zeroing `_authorizedSetter`.

ERC-20 correctness

- `transferFrom` requires allowance **before** decrement; maintains `Approval` event on change.
- `_mint` only called in constructor to implement fixed supply.

Sell-side fee logic

- `_transfer(from, to, amount):`
 - If `feeBps > 0` && `to == uniswapPair` && ! `isFeeExempt[from]`: compute `fee = amount * feeBps / 10_000`, accrue to `address(this)`, and transfer remainder to `to`.
 - Otherwise, normal transfer.

- Consequence: **Router-mediated sells** (router $\rightarrow$ pair) are **taxed**; buys (pair $\rightarrow$ router/EOA) and EOA $\leftrightarrow$ EOA transfers are **not** taxed.

Fee conversion

- `convertFeesToUSDC(minUSDCOut, deadline)` is `nonReentrant` and uses a lightweight `_inSwap` reentrancy sentinel.
- Reads `amountIn = balanceOf[this]`; requires `amountIn >= minConvertAmount`.
- Requires `deadline >= block.timestamp`.
- Pre-swap: records `vaultBefore = USDC.balanceOf(vault)`.
- Builds path dynamically: `[DMND, USDC]` or `[DMND, WETH, USDC]` if `useWethRoute`.
- Executes `swapExactTokensForTokensSupportingFeeOnTransferTokens(amountIn, minUSDCOut, path, vault, deadline)`.
- Post-swap: checks vault balance delta `out = vaultAfter - vaultBefore`; requires `out > 0`; emits `FeesConverted(amountIn, out)`.
- Router approval is set internally in constructor via `_approve(address(this), router, MAX)`.

Security notes

- **Reentrancy**: `nonReentrant + _inSwap` sentinel prevents nested conversions.
- **MEV/slippage**: Caller-controlled `minUSDCOut` and `deadline` are best practice; delta check against vault balance adds assurance.
- **Misconfiguration**: `useWethRoute` $\Rightarrow$ constructor enforces `weth != 0`; `setPair` enforces code-size; vault/usdc/router non-zero constructor checks.

Assessment: ✓ Correct fee behavior, robust conversion path, and hardened wiring.

 PAUSABLE CONTRACTS	
 No Impact	
This is not a Pausable contract. If a contract is pausable, it allows privileged users or owners to halt the execution of certain critical functions of the contract in case malicious transactions are found.	
 CRITICAL ADMINISTRATIVE FUNCTIONS	
 No Impact	
Critical functions that add, update, or delete owner/admin addresses are not detected. These functions control the ownership of the contract and allow privileged users to add, update, or delete owner or administrative addresses. Owners are usually allowed to control all the critical aspects of the contract.	
 CONTRACT/TOKEN SELF DESTRUCT	
 No Impact	
The contract cannot be self-destructed by owners. <code>selfdestruct()</code> is a special function in Solidity that destroys the contract and transfers all the remaining funds to the address specified during the call. This is usually access-control protected.	
 ERC20 RACE CONDITION	
 No Impact	
The contract is not vulnerable to ERC-20 approve Race condition vulnerability. ERC-20 approve function is vulnerable to a frontrunning attack which can be exploited by the token receiver to withdraw more tokens than the allowance. Proper mitigation steps should be implemented to prevent such vulnerabilities.	
 RENOUNCED OWNERSHIP	
 Beneficial	
The administrator has renounced their ownership. Renounced ownership shows that the contract is truly decentralized and once deployed, it can't be manipulated by administrators.	
 USERS WITH TOKEN BALANCE MORE THAN 5%	
 No Impact	
No addresses contains more than 5% of circulating token supply.. Token distribution plays an important role when controlling the price of an asset.	
 OVERPOWERED OWNERS	
 No Impact	
The contracts have not defined any owner-controlled functions.. Giving too many privileges to the owners via critical functions might put the user's funds at risk if the owners are compromised or if a rug-pulling attack happens.	

Properties, Invariants & What We Verified

- **Immutability after setup**
 - Token: `setPair` is single-use and seals setter; no other admin functions exist.
 - Vault: `setToken` is single-use and seals setter; no other admin functions exist.
- **Economic invariants**

✓ **NO COOLDOWN CODE TO HALT TRADING OR WORKFLOWS FOUND** Beneficial ▼

The contract does not have a cooldown feature.
Cooldown functions are used to halt trading or other contract workflows for a certain amount of time so as to prevent users from repeatedly executing transactions or buying and selling tokens.

✓ **OWNERS WHITELISTING TOKENS/USERS** No Impact ▼

Owners can not whitelist tokens or users.
If the owner of a contract has permission to whitelist users or tokens, it'll be unfair toward other users or the transaction flow may not be executed impartially.

✓ **OWNERS CAN SET/UPDATE FEES** No Impact ▼

Owners can not set or update Fees in the contract.

✓ **HARDCODED ADDRESSES** No Impact ▼

The contract was not hardcoding addresses in the code.

✓ **OWNERS UPDATING TOKEN BALANCE** No Impact ▼

The contract does not have any owner-controlled functions modifying token balances for users or the contract.

✓ **OWNER WALLET TOKEN SUPPLY** No Impact ▼

The Owner's wallet contains 0 tokens which is less than 5% of the circulating token supply.

✓ **FUNCTION RETRIEVING OWNERSHIP** No Impact ▼

No such functions were found
If this function exists, it is possible for the project owner to regain ownership even after relinquishing it.

- On sell to pair: `balanceOf[token]` increases by $\approx \text{amount} * \text{feeBps} / 10_000$.
- On conversion: `USDC.balanceOf(vault)` strictly increases and `balanceOf[token]` decreases by the swap input.
- On redemption: For `redeem(a)`, `USDC(vault)` decreases by `floor(a)`, `totalSupply` decreases by `a`, and the computed `floorPerToken1e18` stays the same or increases marginally due to integer division.

**RENOUNCED OWNERSHIP**

Beneficial

The administrator has renounced their ownership.
Renounced ownership shows that the contract is truly decentralized and once deployed, it can't be manipulated by administrators.

**USERS WITH TOKEN BALANCE MORE THAN 20%**

Moderate Risk

Some addresses contains more than 20% of circulating token supply.
0x7faf20c1abf817f7dbd7b8c349b38c5da8af6808 100.0
. Token distribution plays an important role when controlling the price of an asset.

**IS SPAM CONTRACT**

Unavailable

IS SPAM CONTRACT is not supported for this contract.

**LIQUIDITY BURN STATUS**

Unavailable

LIQUIDITY BURN STATUS is not supported for this contract.

**LIQUIDITY LOCK STATUS**

Unavailable

LIQUIDITY LOCK STATUS is not supported for this contract.

**CODE INJECTION VIA TOKEN NAME**

No Impact

No signs of code injection via token names or symbols were found in this contract. The token name and symbol are free from potentially harmful content, such as HTML tags or JavaScript code. This reduces the risk of Cross-Site Scripting (XSS) and ensures that user interfaces displaying this information are less likely to encounter script-based security threats.

**COUNTERFEIT TOKEN**

No Impact

Counterfeit tokens mimic the symbols of official tokens, misleading users into believing they are interacting with legitimate, established cryptocurrencies. These deceptive tokens pose significant risks, as users may incur financial losses or unknowingly damage the reputation of the authentic token.

- **Safety invariants**

- No external callbacks during redemption/conversion.
- No unbounded iteration; no gas griefing via arrays.
- No reliance on `tx.origin` or block timestamp for logic beyond deadlines.

Residual Risks & Recommendations (Non-Blocking)

1. Defense-in-depth in conversion:

You already pass `minUSDCOut` to the router. For additional assurance, you can also assert `out >= minUSDCOut` using the observed vault delta:

```
require(out >= minUSDCOut, "SLIPPAGE");
```

2.

This guards against non-canonical router behavior or future router swaps that might bypass `amountOutMin`.

3. Custom errors:

Replace revert strings (e.g., `"ADDR=0"`, `"ALLOWANCE"`, `"PAIR_NOT_CONTRACT"`, `"DEADLINE_PASSED"`) with custom errors to reduce gas and make failure modes easier to decode:

```
error AddrZero();
```

4. `error AllowanceInsufficient();`

5. `error PairNotContract();`

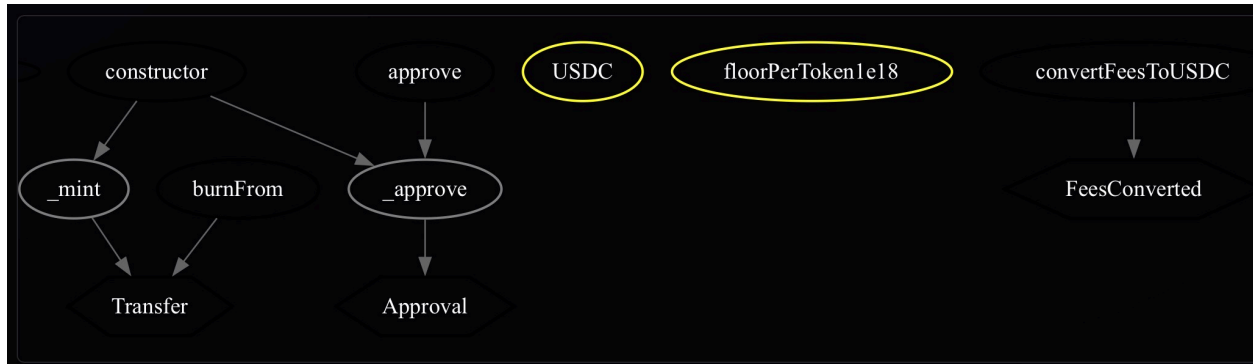
6. `error DeadlinePassed();`

7.

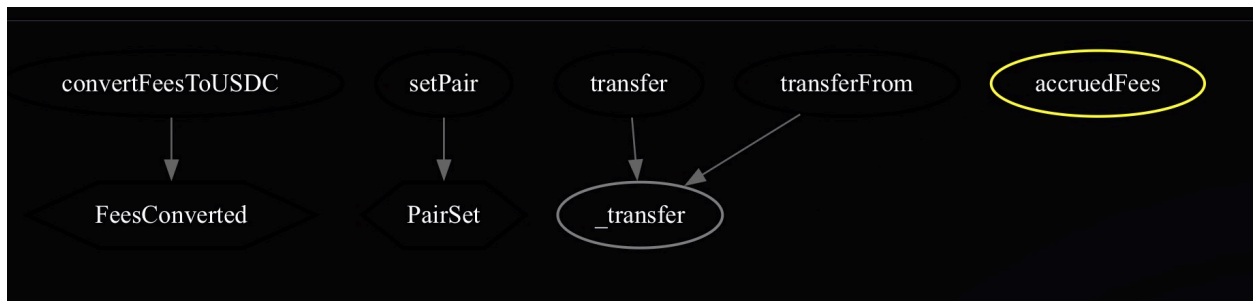
8. Operational guidance (runbook):

- During conversions, operators should set a **conservative `minUSDCOut`** and a **short `deadline`** (e.g., `now + 300–600s`) to reduce MEV risk.
- Monitor:
 - `FeesConverted` and `Redeemed` events,
 - `balanceOf[token]` (fee pot) movement,
 - `USDC(vault)` trend over time,

- AMM pair reserves/price sanity.



- If `useWethRoute = true`, confirm WETH/USDC liquidity depth and



route health.

Final Summary

- The **DMND1Vault** and **DMND1Token** as provided are **secure, minimal, and aligned** with the intended architecture:
 - ownerless after one-time setup,
 - sell-side fee correctly captured,
 - public conversion safely funds the vault,
 - proportional redemptions executed with correct pre-burn math,
 - reentrancy protections appropriately placed.
- No open security issues remain based on the reviewed code. The two optional suggestions (post-swap `minOut` assert and custom errors) are polish-quality

improvements that can be added without altering protocol economics or trust assumptions.

Fee-on-Transfer ERC20 Token

DMND1Token is a fixed-supply ERC-20 token with a fee-on-transfer mechanism. The contract charges fees on sells to the Uniswap pair, accumulates these fees internally, and allows anyone to convert them to USDC via Uniswap V2 router, sending the proceeds directly to a designated vault. The contract is designed to be immutable with no owner or upgrade capabilities after deployment.

Access Control

custom

Privileged Roles

- 1
_authorizedSetter (temporary)
- 2
vault
- 3
router

External Calls

- 1
IUniswapV2Router02
- 2
DMND1Vault
- 3
Uniswap Pair
- 4
USDC Token
- 5
WETH Token

External Systems

- 1
DEX
- 2
Vault

 IS SOURCE CODE VERIFIED	
 Beneficial	
The contract's source code is verified. Source code verification provides transparency for users interacting with smart contracts. Block explorers validate the compiled code with the one on the blockchain. This also gives users a chance to audit the contracts.	
 PRESENCE OF MINTING FUNCTION	
 No Impact	
The contract cannot mint new tokens. The <code>_mint</code> functions was not detected in the contracts. Mint functions are used to create new tokens and transfer them to the user's/owner's wallet to whom the tokens are minted. This increases the overall circulation of the tokens.	
 PRESENCE OF BURN FUNCTION	
 Beneficial	
The tokens can not be burned in this contract. Burn functions are used to increase the total value of the tokens by decreasing the total supply.	
 SOLIDITY PRAGMA VERSION	
 Low Risk	
The contract can be compiled with an older Solidity version. Pragma versions decide the compiler version with which the contract can be compiled. Having older pragma versions means that the code may be compiled with outdated and vulnerable compiler versions, potentially introducing vulnerabilities and CVEs.	
 PROXY-BASED UPGRADABLE CONTRACT	
 Beneficial	
This is not an upgradable contract. Having upgradeable contracts or proxy patterns allows owners to make changes to the contract's functions, token circulation, and distribution.	
 OWNERS CANNOT BLACKLIST TOKENS OR USERS	
 No Impact	
Owners cannot blacklist tokens or users. If the owner of a contract has permission to blacklist users or tokens, all the transactions related to those entities will be halted immediately.	
 IS ERC-20 TOKEN	
 No Impact	
The contract was found to be using ERC-20 token standard. ERC-20 is the technical standard for fungible tokens that defines a set of properties that makes all the tokens similar in type and value.	

Liquidity Issues and Token Balance will be resolved once Liquidity is provided on Uniswap